CiT Backend Engineering Track

Week 17: Spring Kafka & Reliable Messaging

JSON Events, Retries, Dead-Letter Topics, Idempotent Consumers & the Notification Service

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 2 — Specialisation (Weeks 9–20)
Week	Week 17
Instructor	Jim & Anderson
Duration	6 months | 24 weeks
Tech Stack	Java 25, Spring Boot 4, Spring Kafka, Apache Kafka, Docker, PostgreSQL

Confidential — For CiT Trainees Only

Opening Prayer

We begin, as always, by acknowledging that every gift of understanding comes from God. Please bow your heads as we invite His presence into this session.

Prayer of Dedication

Heavenly Father,

Thank You for every lesson You have brought us through, and for the people learning beside us. We confess that in our studies, our work, and our lives, things do not always go to plan. Code breaks, plans fall through, and sometimes we grow tired of trying again.

As Galatians 6:9 encourages us, "And let us not grow weary while doing good, for in due season we shall reap if we do not lose heart." Lord, when an error will not go away or progress feels slow, give us the patience to try again and the humility to ask for help.

Help us build steadily, one small piece at a time, trusting that no faithful effort is wasted. May our work be dependable, our spirits steady, and everything we build a service to others.

In Jesus' name we pray,
Amen.

Scripture for the Week

"And let us not grow weary while doing good, for in due season we shall reap if we do not lose heart."

— Galatians 6:9 (NKJV)

Table of Contents

1. Welcome to Week 17
2. Sending & Receiving JSON Events
3. When Consumers Fail
4. Retries with Back-off
5. Dead-Letter Topics
6. Idempotent Consumers
7. Publishing Reliably — Acknowledgements & the Outbox
8. Module Practical — The Event-Driven Notification Service
9. Key Takeaways, Homework & Exercises

1. Welcome to Week 17

Last week you learned why a request should do only the essential work and announce the rest as events; how Kafka stores events in topics split into partitions, with offsets as bookmarks; how keys keep related events in order; and how consumer groups share the work. You sent String events with KafkaTemplate, read them with @KafkaListener, and attached an X-Correlation-Id header for tracing.

That was the happy path. This week completes the Messaging module by asking the harder question: what happens when something goes wrong? A consumer crashes. A message contains nonsense. The same event arrives twice. A loan is saved but its event never reaches Kafka. Professional systems are designed for all of these — and by the end of the week, yours will be too.

What to Expect in Week 17

By the end of this session, you will be able to:

- Model events as Java records and convert them to and from JSON with Jackson
- Tell transient failures from permanent ("poison") ones, and explain delivery guarantees
- Configure retries with back-off using a DefaultErrorHandler
- Route messages that keep failing to a dead-letter topic, then inspect and replay them
- Write idempotent consumers that are safe when an event arrives twice
- Explain the dual-write problem and how the transactional outbox solves it
- Build an event-driven notification service — the Messaging module practical

2. Sending & Receiving JSON Events

In Week 16 your event was a sentence like "Member 42 borrowed Things Fall Apart". People can read that, but programs cannot reliably pull facts out of it. Real events carry structured data as JSON.

2.1 Model the Event as a Record

A Java record (Week 5) suits an event perfectly: a fact should never change once created. Here is last week's event shape as Java:

```
public record LoanCreatedEvent(
        UUID eventId,
        String eventType,          // "LOAN_CREATED"
        Instant occurredAt,        // always UTC
        LoanCreatedData data) {
}

public record LoanCreatedData(Long loanId, String memberId, Long bookId, LocalDate
dueDate) {
}
```

Your Java components stay camelCase. Because of the `SNAKE_CASE` naming setting you added in Week 10, the JSON comes out as `event_id`, `occurred_at`, and `due_date` — the same convention as your API.

2.2 Convert Explicitly with Jackson

Keep the String serializers from Week 16 and convert the record yourself. Spring Boot 4 uses Jackson 3 and already provides a configured `JsonMapper` bean, so simply inject it:

```
@Component
public class LoanEventProducer {

    private final KafkaTemplate<String, String> kafkaTemplate;
    private final JsonMapper jsonMapper;

    // constructor injection of both beans, as in Week 9

    public void loanCreated(LoanCreatedEvent event) {
        String payload = jsonMapper.writeValueAsString(event);   // record -> JSON text
        kafkaTemplate.send("library.loan-created", event.data().memberId(), payload);
```

```
    }
}
```

On the consuming side, turn the text back into a record:

```
@KafkaListener(topics = "library.loan-created", groupId = "notification-service")
public void onLoanCreated(String message) {
    LoanCreatedEvent event = jsonMapper.readValue(message, LoanCreatedEvent.class);
    log.info("Loan {} is due on {}", event.data().loanId(), event.data().dueDate());
}
```

Why Convert by Hand?

Spring Kafka can also convert JSON for you with special serializers. We convert explicitly so you see exactly what crosses the wire: a plain string of JSON that any language can read.

Note that Jackson 3 exceptions are unchecked: `readValue` does not force a try/catch on you. If the message is not valid JSON, it simply throws — and Section 3 is about what happens next.

Exercise 1: Your First JSON Event

1. Create the LoanCreatedEvent and LoanCreatedData records in your Library API's messaging package
2. Have the facade build the event after saving a loan, with a random UUID and Instant.now()
3. Borrow a book in Postman and check the console consumer shows snake_case JSON
4. In the listener, log the loan id and due date from the parsed record

3. When Consumers Fail

Before fixing failures, understand them. The right response depends on the kind of failure.

3.1 Transient vs Permanent Failures

Kind	What It Looks Like
Transient	Temporary: the database restarted, the network blinked, the SMS gateway timed out. Trying again in a moment will probably work.
Permanent ("poison")	The message itself is the problem: broken JSON, a missing field, a loan id that does not exist. Trying again will fail exactly the same way, forever.

The Boda-Boda Delivery

A boda-boda rider finds the gate locked — transient. He waits a few minutes and tries again. But if the address does not exist — permanent — no waiting helps. He brings the parcel back to the office for someone to sort out and carries on with other deliveries. Your consumer needs the same judgement.

3.2 How Spring Commits Offsets

Recall that a consumer group remembers its position with an offset. Spring commits that offset only after your listener method returns successfully. That single fact explains a lot:

- If the listener throws, the offset is not committed and the record can be delivered again
- If the service crashes after the work but before the commit, the event arrives again after restart
- So duplicates are possible, and your consumer must expect them (Section 6)

The Default Is Not Enough

Without any configuration, Spring retries a failing record a few times in quick succession and then logs the error and moves on to the next record. The failed event is effectively lost — no SMS, no record, only a log line nobody may read. The next two sections replace this with a deliberate policy.

3.3 Delivery Guarantees

Guarantee	Meaning	Trade-off
At-most-once	Each event is processed once or not at all	No duplicates, but events can be lost
At-least-once	Each event is processed one or more times	Nothing lost, but duplicates happen — the usual choice
Exactly-once	Each event has its effect exactly once	Hard and only within limits; in practice we get it with at-least-once plus idempotent consumers

Exercise 2: Transient or Poison?

Classify each failure and say what the consumer should do:

5. The PostgreSQL connection was refused for 20 seconds during a restart
6. The message is {"loan_id": "abc"} — the id is not a number
7. The SMS gateway returned "503 Service Unavailable"

4. Retries with Back-off

Transient failures usually heal themselves within seconds. The answer is to wait a little and try again — and to stop after a sensible number of attempts.

4.1 A DefaultErrorHandler Bean

Spring Kafka hands every failed record to an error handler. Declare your own DefaultErrorHandler in the configuration package:

```
@Configuration
public class KafkaErrorHandlingConfig {

   @Bean
   DefaultErrorHandler kafkaErrorHandler() {
      // wait 1 second between attempts; retry 2 times after the first try
      return new DefaultErrorHandler(new FixedBackOff(1000L, 2L));
   }
}
```

A failing record is now tried three times in total: the first attempt plus two retries, one second apart. Spring Boot applies a single error handler bean like this to the listener container behind your @KafkaListener methods — no extra wiring needed.

4.2 Exponential Back-off

A fixed pause is fine for short blips. For a service that may need longer to recover, ExponentialBackOff doubles the wait each time — 1 second, 2, 4, 8 — giving the other system breathing room instead of hammering it.

```
ExponentialBackOff backOff = new ExponentialBackOff(1000L, 2.0);   // start at 1s, double
backOff.setMaxElapsedTime(15000L);                                 // give up after ~15s
return new DefaultErrorHandler(backOff);
```

Retries Block the Partition

While one record is being retried, the records behind it in that partition wait. Keep the total retry time short — seconds, not minutes. For longer waits, Spring Kafka offers @RetryableTopic, which moves a failed record to separate retry topics so the main topic keeps flowing (non-blocking retries).

4.3 Do Not Retry Poison Messages

Retrying a broken message only wastes time. Tell the handler which exceptions are permanent so it gives up immediately:

```
DefaultErrorHandler handler = new DefaultErrorHandler(new FixedBackOff(1000L, 2L));
handler.addNotRetryableExceptions(
      InvalidEventException.class,    // our own validation failure
      JacksonException.class);        // the message is not valid JSON
return handler;
```

InvalidEventException is your own unchecked exception for an event that fails a check, such as a missing loan_id.

Exercise 3: Watch the Retries

8. Add the KafkaErrorHandlingConfig above to the consumer side of your Library API
9. Make the listener throw a RuntimeException when the loan id is even
10. Borrow two books and read the logs: count the attempts and the gap between them
11. Switch to InvalidEventException and confirm it is no longer retried

5. Dead-Letter Topics

When retries run out, or a message is poison, it must not vanish. It goes to a dead-letter topic (DLT) — the parcel brought back to the office for a person to examine.

5.1 Publishing to the DLT

Give the error handler a DeadLetterPublishingRecoverer. A recoverer is what runs once retrying is over:

```
@Bean
DefaultErrorHandler kafkaErrorHandler(KafkaTemplate<String, String> kafkaTemplate) {
   DeadLetterPublishingRecoverer recoverer = new
DeadLetterPublishingRecoverer(kafkaTemplate);
   DefaultErrorHandler handler = new DefaultErrorHandler(recoverer, new
FixedBackOff(1000L, 2L));
   handler.addNotRetryableExceptions(InvalidEventException.class,
JacksonException.class);
   return handler;
}
```

By default the recoverer publishes the failed record to a topic named <original-topic>.DLT, on the same partition number as the original. So declare the DLT with the same partition count as the main topic:

```
@Bean
NewTopic loanCreatedDlt() {
   return TopicBuilder.name("library.loan-created.DLT").partitions(3).replicas(1).build();
}
```

The recoverer also adds headers explaining the failure, such as kafka_dlt-exception-message, kafka_dlt-original-topic, and kafka_dlt-original-offset.

5.2 Monitor, Inspect, Fix, Replay

12. **Monitor** — a message in a DLT means something needs attention. A small listener on the DLT that logs at ERROR level lets your alerting notice.
13. **Inspect** — read the message and its headers to learn why it failed:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh \
 --bootstrap-server localhost:9092 --topic library.loan-created.DLT \
 --from-beginning --property print.headers=true
```

14. **Fix** — correct the bug in the consumer, or the bad data at its source.
15. **Replay** — re-publish the dead-lettered messages to the original topic with a small admin tool or script.

Never Silently Swallow Failures

It is tempting to wrap a listener in `try { ... } catch (Exception e) { }` so errors "go away". They do not go away — a member simply never gets their reminder and nobody ever knows why. Let exceptions reach the error handler, so every failure is retried or parked in the DLT where a person will see it.

Exercise 4: Send a Poison Message

16. Add the recoverer and declare library.loan-created.DLT with 3 partitions
17. With the console producer, send the text "this is not json" to library.loan-created
18. Confirm from the logs that it was not retried, then find it in the DLT with its headers
19. Write two sentences for your team explaining how they would replay it

6. Idempotent Consumers

With at-least-once delivery, the same event will sometimes arrive twice. The consumer must make that harmless.

6.1 What Idempotent Means

An operation is idempotent if doing it twice has the same effect as doing it once. Pressing a lift button five times still brings one lift. Some operations are naturally idempotent; others are not:

Naturally idempotent	Not idempotent
Set loan status = RETURNED	Add 1 to the member's "books borrowed" counter
Save the loan with id 1042 (upsert)	Insert a new row for every event received
Set the balance to UGX 50,000	Add UGX 50,000 to the balance

Prefer the left column. When you cannot, remember which events you have handled.

6.2 A processed_events Table

```
CREATE TABLE processed_events (
    event_id     UUID PRIMARY KEY,
    processed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
@KafkaListener(topics = "library.loan-created", groupId = "notification-service")
@Transactional
public void onLoanCreated(String message) {
    LoanCreatedEvent event = jsonMapper.readValue(message, LoanCreatedEvent.class);

    // processedEvents is a JpaRepository<ProcessedEvent, UUID>
    if (processedEvents.existsById(event.eventId())) {
        log.info("Duplicate event {} skipped", event.eventId());
        return;                                  // already done
    }
```

```
    loanViewService.record(event);                  // the real work
    processedEvents.save(new ProcessedEvent(event.eventId()));
}
```

What Just Happened

The work and the "I have processed this" record are saved in one database transaction: both happen, or neither does. If the event arrives again, the check finds it and returns. If two copies arrive at the same moment, both may pass the check — but the primary key lets only one insert succeed. The other transaction rolls back, and on retry the check skips it. The database is your final safety net.

A Transaction Cannot Un-send an SMS

A rollback undoes database changes, not calls to outside systems. If the service sends an SMS and crashes before committing, the retry may send it again. Keep outside calls few, and pass the event id where a provider supports an idempotency key.

Exercise 5: Prove It Is Safe

Create the processed_events table and add the check to your listener. Copy one loan-created JSON message from the console consumer and paste it into the console producer three times. Show from the logs and the database that the work happened once and two duplicates were skipped.

7. Publishing Reliably — Acknowledgements & the Outbox

Now look at the other end: how sure can the producer be that its event reached Kafka at all?

7.1 Producer Acknowledgements

When a producer sends an event, the broker replies with an acknowledgement (ack). The acks setting decides when: acks=all waits until every in-sync copy of the partition has the event — the safest choice. Modern Kafka clients already default to acks=all and to an idempotent producer, which stops a network retry from writing the same event twice. Just do not switch them off.

7.2 The Dual-Write Problem

Look closely at the borrow flow. It writes to two places: PostgreSQL (the loan) and Kafka (the event). These two writes are not one atomic action:

- The loan commits, then Kafka is unreachable — the loan exists but no event is ever sent, so no SMS
- The event is sent, then the database transaction rolls back — consumers hear about a loan that never existed

7.3 The Transactional Outbox

The fix is to make only one write that matters. Instead of sending to Kafka, save the event to an outbox table in the same database transaction as the loan. A separate publisher then sends outbox rows to Kafka.

```
@Transactional
public Loan create(String memberId, Long bookId) {
   Loan loan = loanRepository.save(new Loan(memberId, bookId));
   LoanCreatedEvent event = LoanEvents.created(loan);        // builds the record
   outboxRepository.save(new OutboxMessage("library.loan-created", memberId,
         jsonMapper.writeValueAsString(event)));    // same transaction as the loan
   return loan;
}

@Scheduled(fixedDelay = 2000)                          // every 2 seconds
@Transactional
public void publishPending() {
```

```
    for (OutboxMessage msg :
outboxRepository.findTop100BySentAtIsNullOrderByCreatedAt()) {
        kafkaTemplate.send(msg.getTopic(), msg.getMessageKey(), msg.getPayload()).join();
        msg.markSent();                          // sets sent_at
    }
}
```

Why This Works

The loan and its outbox row commit together or not at all, so an event is never lost or invented. If Kafka is down, rows wait. If the publisher crashes after sending but before marking a row sent, the row is sent again — a duplicate your idempotent consumers already handle. Remember @EnableScheduling on a configuration class.

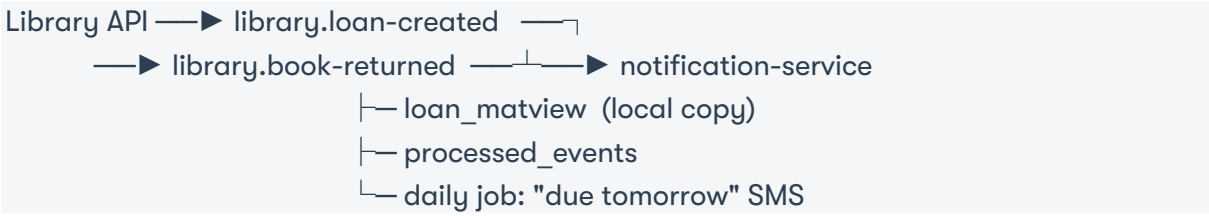
Exercise 6: Spot the Dual Write

A school fees system saves a payment of UGX 450,000, then publishes fees.payment-received directly. Describe one failure that loses the event and one that invents an event. Then sketch the outbox table you would add.

8. Module Practical — The Event-Driven Notification Service

Time to bring the module together: a second, separate Spring Boot application that listens to the Library API's events and sends members reminders.

8.1 The Design



Create the project on start.spring.io with Spring for Apache Kafka, Spring Data JPA, and the PostgreSQL Driver. It has its own database and the group id notification-service. It never calls the Library API: it learns everything from events and keeps its own loan_matview table — on the XENO platform, the _matview suffix marks a local copy maintained by a Kafka consumer.

8.2 Reading the Correlation ID

Receive the whole ConsumerRecord so you can read headers, and put the correlation id into the MDC so every log line carries it:

```
@KafkaListener(topics = "library.loan-created", groupId = "notification-service")
@Transactional
public void onLoanCreated(ConsumerRecord<String, String> record) {
   Header header = record.headers().lastHeader("X-Correlation-Id");
   String correlationId = header != null
         ? new String(header.value(), StandardCharsets.UTF_8)
         : UUID.randomUUID().toString();          // never lose traceability
   MDC.put("correlationId", correlationId);
   try {
      LoanCreatedEvent event = jsonMapper.readValue(record.value(),
LoanCreatedEvent.class);
      if (processedEvents.existsById(event.eventId())) return;
      loanMatviewRepository.save(LoanView.from(event));       // status ACTIVE
      processedEvents.save(new ProcessedEvent(event.eventId()));
      log.info("Loan {} recorded, due {}", event.data().loanId(), event.data().dueDate());
```

```
    } finally {
        MDC.remove("correlationId");          // the thread is reused
    }
}
```

The book-returned listener follows the same pattern and sets the loan's status to RETURNED.

8.3 The Daily Reminder Job

```
@Scheduled(cron = "0 0 9 * * *", zone = "Africa/Kampala")   // 9:00 every morning
public void remindLoansDueTomorrow() {
    LocalDate tomorrow = LocalDate.now(ZoneId.of("Africa/Kampala")).plusDays(1);
    for (LoanView loan : loanMatviewRepository.findByDueDateAndStatus(tomorrow,
LoanStatus.ACTIVE)) {
        smsSender.send(loan.getMemberId(), "Reminder: your library book is due tomorrow.");
    }
}
```

SmsSender is an interface. For this practical, its only implementation logs the message instead of calling a real SMS provider. In a real system it would look up the member's phone number with proper permission — events carry ids, not personal data.

Practical Requirements

20. A separate notification-service app with its own database, consuming JSON loan-created and book-returned events
21. X-Correlation-Id read from every record, placed in the MDC, and cleared afterwards
22. A loan_matview table kept up to date from both events
23. Retries with back-off, no retries for poison messages, DLTs for both topics, and idempotent listeners
24. A daily @Scheduled job that "sends" due-tomorrow reminders through a logging SmsSender
25. A README describing the topics, payloads, consumer group, and how to run it

Exercise 7: Build and Break the Notification Service

Build the service to the requirements above. Create a loan due tomorrow and trigger the job (temporarily, a cron that runs every minute). Then stop the service, borrow three books, and restart it — show that it catches up on every event, with no duplicates and the right correlation ids in the logs.

Module Complete — Messaging & Event-Driven Architecture

Congratulations! In two weeks you have gone from "what is an event?" to a second service that reacts to events reliably: JSON events, retries, dead-letter topics, idempotent consumers, the outbox, and end-to-end correlation ids — the patterns behind real payment and notification systems.

9. Key Takeaways, Homework & Exercises

9.1 What You Learned This Week

- Events as Java records, converted to JSON with the auto-configured JsonMapper
- Transient vs poison failures; at-most-once, at-least-once, and exactly-once delivery
- Retries with FixedBackOff or ExponentialBackOff in a DefaultErrorHandler, skipping retries for permanent errors
- Dead-letter topics: monitor, inspect, fix, and replay — never swallow failures
- Idempotent consumers with a processed_events table protected by a primary key
- Producer acks, the dual-write problem, and the transactional outbox
- A notification service with a _matview table, MDC correlation ids, and a scheduled job

9.2 Homework

26. Complete Exercise 7 and push both the Library API and notification-service to GitHub
27. Add a library.book-returned.DLT topic and a DLT listener that logs failures at ERROR level
28. Optional stretch: move the Library API's loan-created publishing to a transactional outbox
29. Preview Week 18: Read about database indexes and the N+1 query problem, and what Redis is. Come with one question ready.

Coming Next — Week 18: Caching & Performance

The Caching, Performance & DevOps module begins. A correct system must also be fast. You will learn to measure before you optimise, find and fix N+1 queries, speed up slow lookups with database indexes, and keep frequently read data close at hand with Redis and Spring Cache.

Closing Reflection

"For a righteous man may fall seven times and rise again."

— Proverbs 24:16 (NKJV)

This week you built systems that expect to fail and are designed to get back up: a retry tries again, and a dead-letter topic keeps what could not be handled until

someone can help. Your learning works the same way. You will fall — a build that will not pass, a concept that will not click. Falling is not the end of the story. Rise again, ask for help, and keep going.

Glossary of Terms

Term	Definition
Acknowledgement (acks)	The broker's confirmation that it has stored an event; acks=all waits for every in-sync copy
At-Least-Once	A delivery guarantee where nothing is lost but an event may be processed more than once
Back-off	A pause between retry attempts, fixed or growing each time
Dead-Letter Topic (DLT)	A topic where messages that could not be processed are parked for inspection
DefaultErrorHandler	The Spring Kafka class that retries failed records and then hands them to a recoverer
Dual-Write Problem	Writing to a database and to Kafka as two separate steps, so one can succeed while the other fails
Idempotent	Having the same effect whether it is done once or many times
JsonMapper	The Jackson 3 object that converts between Java objects and JSON text
MDC	Mapped Diagnostic Context: values such as a correlation id added to every log line on a thread
Outbox	A table where events are saved in the same transaction as the data, then published separately
Poison Message	A message that will fail every time it is processed, however often it is retried
Recoverer	What the error handler runs when retries are exhausted, such as publishing to a DLT
Transient Failure	A temporary failure that will probably succeed if tried again shortly
_matview	XENO naming suffix for a local, read-only copy of another service's data kept up to date by a consumer